Web Accessibility

These slides are modified from slides taken from www.csuci.edu by Peter Mosinskis

Overview

- The User Experience
- Makeover techniques to fix Web Accessibility issues
- Accessibility checking
- Useful links

The User Experience

"Keeping Web Accessibility In Mind"

Video available online at:
www.webaim.org

Types of Disabilities

- Visual (blindness, low vision, color-blindness)
- Hearing (deafness, hard of hearing)
- Physical/Motor (weakness, muscle control, paralysis)
- Cognitive/Neurological (dyslexia, intellectual or memory impairments)
- Multiple Disabilities (deaf and blind)

Some scenarios

- online shopper with color blindness
- reporter with repetitive stress injury
- online student who is deaf
- accountant with blindness
- classroom student with dyslexia
- retiree with aging-related conditions, managing personal finances
- supermarket assistant with cognitive disability
- teenager with deaf-blindness, seeking entertainment

What do these means to designers?

- Disability accommodations must be provided quickly and **as needed**
- Technology, web sites and any electronic documents must be accessible
  from their inception, not ad hoc

Benefits

- *Scalability*: do it once rather than coordinating accommodations each and every time
- *Reduces cost* for retrofitting
- *Greater return on investment*: non-disabled students will benefit from universal design
  - Universal Design = Access for All
  - Multiple means of access to information
  - Way to reach “digital generation”
- *Manage Risk* (the reactive model often does not result in timely or “as effective as” access)

Accessibility

Most Frequent Checkpoints

(1) Text description of non-text elements

- First determine if each image is **meaningful** or **decorative**
- If image is *meaningful*
  - Write short description (7 to 80 characters long)
  - Add ALT attribute & description to <img> element
  - If necessary, add long description using LONGDESC attribute
- If image is *decorative*:
  - Add “empty” ALT attribute of <img> element

Example Image #1 (meaningful)

HTML Code for Example Image #1

- WRONG:
  <img src="soccer.jpg" />
- WRONG:
  <img **alt**="A baked potato"
  src="soccer.jpg" />
- RIGHT:
  <img **alt**="Soccer goalie leaping for ball"
  src="soccer.jpg" />

Example Image #2 (decorative)

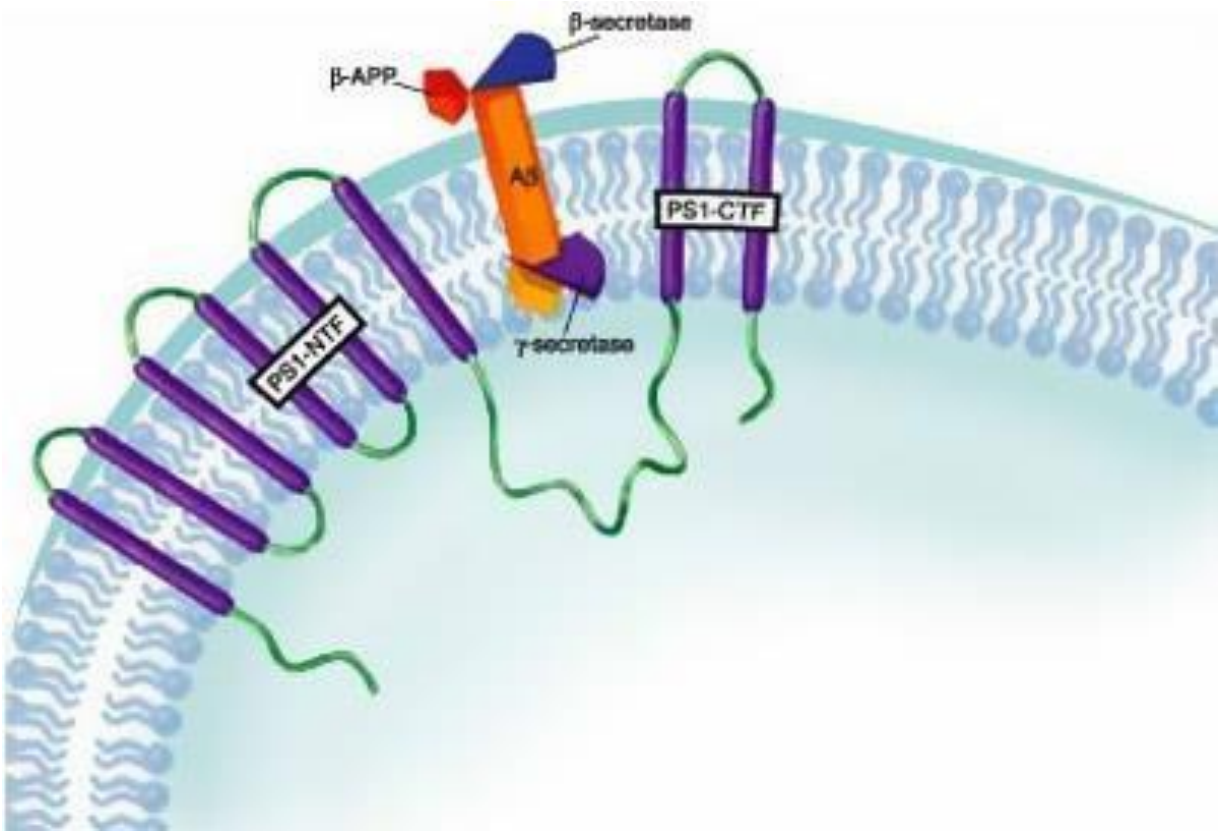
HTML Code for Example Image #2

- WRONG:
  <img src="soccer.jpg" />
- WRONG:
  <img **alt**="Hands on soccer ball" src="soccer.jpg" />
- RIGHT:
  <img **alt**="" src="soccer.jpg" />

Using LONGDESC

- What LONGDESC is
  - An attribute of the <img> element
- When to use it
  - When you need 80+ characters or more to describe an image
  - Probably <5% of all images
- How it’s used
  - <img **alt**=“My short description” **longdesc**=“URL to my description” />

Example Image #3



HTML Code for Example Image #3

- WRONG:

  <img src="cellstructures.jpg" />

- WRONG:

  <img **alt**="Hands on soccer ball" src="cellstructures.jpg" />

- RIGHT:

  <img **alt**="Beta-secretase enzyme" **longdesc**=" http://www.csuci.edu/cs-description.htm" src="cellstructures.jpg" />

Also Applies to Video and Audio

- Video clips must be *captioned*
- Audio-only clips require a text transcript
  - Audio-only clips don't have to be captioned (they are not multimedia)

(2) Content can be conveyed without color

- Two main issues
  - Use of color words
  - Color contrast of foreground/background colors

Issue #1 Color Words

- **Never depend on color alone**: you can use color, but not to define sole source of information or meaning
- WRONG:
  - “Items marked in red must be completed”
  - “Push the green button to start”
- CORRECT:
  - “Items marked with * must be completed”
  - “Push the button titled “Start” to begin”

Issue #1 Color Words

- Need to add emphasis?
  - Use **** or **** to add emphasis instead of color
- Can I still use color?
  - Yes, good use of color = good design

Issue #2 Contrast

- Check contrast of text and background colors
  - Desktop tool: Colour Contrast Analyzer
  - Web based: http://juicystudio.com/services/colourcontrast.php
- Highest contrast color scheme?
  - Black Text, White background!

(3) Document should be readable without style sheet

- Check #1 (Firefox):
  - Web Developer Toolbar >
    CSS button >
    Disable Styles >
    All Styles
- Check #1 (Internet Explorer):
  - Use Web Accessibility Toolbar for IE 2.0
    http://www.paciellogroup.com/resources/wat-ie-about.html

Document should be readable without style sheet

- Check #1 (Firefox):
- Web Developer Toolbar >
  Information button >
  View Document Outline
- Page sections should be displayed in an organized, outlined form

Document should be readable without style sheet

- Why is HTML structure important?
  - Screen reader software reads a page based on the HTML structure, NOT on the visual organization of page content
- Checkpoints:
  - Can the page still be read?
  - Can the page still be navigated?
  - Is the organization of the page still clear and easily comprehensible?

Guidelines for Repair

- Use HTML only for *structure* your content (not to apply formatting)
- Structure your content using headings (<h1>) paragraphs (<p>), lists (<ol>, <ul>, and <li>) and tables
- Use Cascading Style Sheets (CSS) to apply formatting and set the *look* of your page
- Do not use HTML elements for purposes other than what they were intended (Example: use <blockquote> only for quotes, not to indent)

(4)Make tables accessible

- Two common table types
  - Layout tables
    - Invisible
    - Used to design layout for (older) web pages
  - Data tables
    - Used to display a set of data in columns & rows
- Accessibility guidelines apply **only to data** tables
- Check and repair: 4 steps

Sample Table Code

```
<table border="1" summary="Lists names and favorite colors">
<thead>
<tr>
        <th scope="col">Name</th>
        <th scope="col">Favorite Color</th>
</tr>
</thead>
<tbody>
        <tr>
          <td>Pat Smith</td>
          <td>Red</td>
        </tr>
        <tr>
          <td>Tyler Zipper</td>
          <td>Green</td>
        </tr>
</tbody>
</table>
```

Fixing tables – Step 1

- First, remove non-table data from the table

Fixing tables – Step 2

- The CAPTION element can be used to associate a title with the table

<caption>Computer Science Job Openings - November 2011</caption>

Fixing tables – Step 3

- Add a table summary using “summary” attribute
  - Provides a textual description of the table
- Example:
  <table summary=“List of names and favorite colors”>

Fixing tables – Step 4

- Use <th> (table heading) tags for column and row headings
  - First data table row should always contain <th> tags
- Example:
  - **<th** scope="col"**>**Name**</th>**

(5) Applets, plug-ins and external content – Step 1

- Make your PDF, Word, Excel, PowerPoint, etc. accessible
- Similar accessibility techniques apply to these file formats
- Provide links to the software used to view/read the document

(6) Skip Navigation Links

- Each page must have a link that enables a user to jump to the main body content of a page
- Enables skipping of repetitive navigation links

How to Fix:

1. Directly after <body> tag, add the following HTML:

```
<div id="skip">
<a href="#back_to_top" name="back_to_top"
id="back_to_top"></a>
<a href="#skip_navigation"><img
src="http://www.csuci.edu/images/spacer.gif" width="1" height="1"
border="0" alt="Skip Global Navigation"></a>
</div>
```

How to Fix:

2. At the start of the actual page body content, add the following HTML code:

```
<a name="skip_navigation” id="skip_navigation">
</a>
```

Accesibility

Less Frequent Checkpoints

…but no less important…

(1) Multimedia presentation alternatives and synchronization

- Captioning must be done in real-time
- Captioning must be synchronized with presentation (not a transcript provided later)
- Webcasts require real-time captioning
- How to Check:
  - Manual check required
- How to Fix
  - Captioning Software (MAGPie; HiCaption)

(2) Eliminate screen flicker

- Bad examples: http://www.hosanna1.com/
- People with photosensitive epilepsy can have seizures triggered by flickering or flashing
- Some people with cognitive or visual disabilities can't read moving text
- Moving text or images can be distracting
- How to fix:
  - Remove blinking or flashing elements on a web page
  - Remove banner text that automatically scrolls
  - Remove animated GIFs – they are a nuisance

(3) Text-only page

- If accessibility can't be accomplished in any other way, text-only page is a possible alternative
- **Fact**: There are virtually no instances (98%+) where this should be necessary
- **Fact**: Text-only page by itself does not address accessibility (for example, what happens to content structure?)

Text-only page (continued)

- Caveats to text-only page:
  - Text-only page must be updated whenever primary page changes
  - Text-only page must have equivalent information and functionality
- How to Fix
  - Design (or redesign) carefully with accessibility in mind, not as an afterthought

(4) Accessible Forms

- Information, field elements, and functionality of HTML forms must be accessible, including directions & cues
- Users should be able to complete and submit the form using assistive technology

Accessible Forms

- Form elements which need to be accessible
  - Push buttons
    <INPUT type="button“> or <INPUT type="submit“>
  - Image buttons
    <INPUT type="image"> and <BUTTON>
  - Text entry fields
    <INPUT type="text"> and <TEXTAREA>
  - Radio buttons <INPUT type="radio">
  - Check boxes <INPUT type="checkbox">
  - Select (drop-down or list) menus <SELECT>

Accessible Forms

- Push buttons: add “value” attribute
  - <input type="submit" **value=“Submit This Form"**>
- Image buttons: add “alt” attribute
  - <input type="image" name="Go" src="go.gif" **alt="Go"**>
- Text Entry Fields:
  - Add “id” attribute and value to <input> tag
  - Add <label> tag **before** a text field
  - Add “for” attribute and value to <label> tag; set to “id” value of <input> tag
  - **Example:**
    <label for=“firstname”>First Name:</label><input type="text" **id=“firstname”** name=“fn" size=20>

Accessible Forms

- Radio buttons & Check Boxes
  - Add <label> right **after** <input> tag
  - Example:
    <input type="radio" id="age01" value="Under 40"><label for="age01">Under 40</label>

Accessible Forms

- Select menus
  - Add <label> right **before** <select> tag
  - Example:
    <label for="favcolor">Favorite Color</label>
    <select title="Choose Your Favorite Color"
    name="favoritecolor" **id="favcolor"**>
    <option value="">Please select one</option>
    <option value="">Blue</option>
    <option value="">Red</option>
    </select>

Accessible Forms

- How to Check
  - Can you navigate the form using your keyboard?
- Tips for Accessible Forms
  - Use <label> tag when possible
  - Add “title” attribute to <input>, <select>, and <textarea> tags
  - Use <fieldset> to group sets of form elements
  - Use <legend> to provide information on form elements
  - Use <optgroup> to group items in <select> lists
- General Tips for Enhancing Web Forms
  - Using <label> tags you can move around where labels are placed (e.g., label in one cell of table, form field in another)
  - Apply style to forms using CSS

Accessibility

Least Frequent Checkpoints

…but still required…

(1) Image Maps

- Image maps let you create clickable, linked areas on an portion of an image in a web page
- You must provide links in your HTML code to all linked map areas for all server-side image maps
- Use client-side image maps instead of server-side image maps

Image Maps

- **How to Fix:**
  - Don’t use Server Side image maps (they are deprecated).
  - Reconfigure Server Side image maps as client-side image maps
  - Use ALT text on your image map link areas

Scripts and functional text

- Any JavaScript should have “functional text”
  - “Functional text” is text that identifies what will happen when a script is activated.
- How to Test:
  - Test JavaScript events without a mouse (keyboard only)
  - Test your site on a screen reader or talking browser
  - Turn JavaScript off, test all elements on page being affected by JavaScript to ensure no information is lost

Web Content Accessibility Guidelines (WCAG)

The Web Content Accessibility Guidelines (WCAG) are a set of international standards for making web content more accessible to people with disabilities.

Developed by the World Wide Web Consortium (W3C) through its Web Accessibility Initiative (WAI), WCAG aims to provide a single shared standard for web content accessibility that meets the needs of individuals, organizations, and governments internationally.

WCAG Versions

- **WCAG 1.0**: Released in 1999, it laid the foundation for web accessibility but was later found to be somewhat limited in its scope.

- **WCAG 2.0**: Published in 2008, it expanded and refined the guidelines, making them technology-neutral and more flexible.

- **WCAG 2.1**: Introduced in 2018, it added additional success criteria to address mobile accessibility, people with low vision, and people with cognitive and learning disabilities.

- **WCAG 2.2**: As of 2023, the latest draft, builds on WCAG 2.1 by introducing new success criteria and enhancements.

Link: https://wcag.com/resource/what-is-wcag/

Key Principles of WCAG

WCAG is based on four key principles, often abbreviated as **POUR**:

- **Perceivable**: Information and user interface components must be presented in ways that users can perceive.
- **Operable**: User interface components and navigation must be operable.
- **Understandable**: Information and the operation of the user interface must be understandable.
- **Robust**: Content must be robust enough that it can be reliably interpreted by a wide variety of user agents, including assistive technologies.

Implementation and Benefits

Implementing WCAG guidelines ensures that websites and web applications are accessible to a wider audience, including those with disabilities:

- **Legal Compliance**: Many regions have legal requirements for web accessibility.
- **Improved User Experience**: Accessible design often leads to better usability for all users.
- **Market Reach**: Access to a broader audience, including people with disabilities.
- **SEO Benefits**: Some accessibility practices can improve search engine optimization.

Timed Response

- When a timed response is required, the user shall be alerted and given sufficient time to indicate more time is required.
- How to check:
  - Manual check - does your page require a timed response to a form?
- How to fix:
  - Your web application must have an alerting mechanism so users can request more time

Now, Check your work

- No tool can test all Web standards automatically
- A list of tools available at: http://www.w3.org/WAI/RC/tools/complete
- Verification requires use of automated checkers, assistive technology and manual checks
  - First run an automated checking tool and fix any errors identified in reports
  - Listen with assistive technology to ensure the site is accessible and usable
    - Logon pages, templates, navigation
    - Pages with data tables, forms and multimedia
  - Verify keyboard access
    - F6 to frames
    - Tab to links and controls (e.g. form input fields)
    - JavaScript keyboard access
  - Additional manual checks for some standards
    - JavaScript
    - Color
    - Multimedia

Always ask yourself:

- Can all information be accessed by every user?
- Can all information be searchable?

and:

- Have I added necessary comments?

Useful links

- WCAG Standards - https://www.w3.org/WAI/standards-guidelines/wcag/
- Web Standards Project: http://www.webstandards.org/learn/
- WebAIM - http://www.webaim.org/